Certified Theorem Subsumption in Lean 4

Design for Weaker Requirements, Stronger Guarantees, and Certified Adapters

Scope. The design distinguishes strictness from one-way provability, separates context-free guarantee strength from contextual adaptation, excludes blind whole-theorem implication from the default classifier, makes the proof background explicit, and requires closed kernel-checked certificates.

Proof-of-concept design for `#compare_theorems`

Lean 4 metaprogramming, proof automation, dependency tracking, and domain-specific semantic comparison

October 2026

Implementation status

This is a design specification. It does not, by itself, establish that a repository currently implements the command or that existing theorem selections were produced by this criterion. Any implementation claim should be backed by generated obligations, closed certificates, dependency data, and a selection manifest.

Executive Summary

Theorem generality can be made precise by generating explicit Lean proof obligations. Positive judgements are sound when the generated proposition is explicit and the resulting proof is closed and kernel-checked. The comparator is therefore an obligation generator plus a bounded proof-search and certification pipeline, not a semantic oracle.

Three distinctions are essential. First, proving one direction while failing to prove the reverse does not establish strictness: reverse failure is UNKNOWN unless the reverse is actually refuted. Second, a conclusion implication proved in the target requirement context is a contextual adapter obligation, not the same thing as context-free guarantee strength. Third, blindly asking Lean to prove `T1 -> T2` in a library that already contains `T2` is vacuous, because the proof can simply use the target theorem.

The design therefore makes the comparison mode, alignment, proof background, generated obligations, and dependencies part of every result. It also supports domain-specific comparators for cases such as numerical upper bounds, where comparing the promised bounds themselves is more informative than comparing already-proved propositions.

Design principles

- **Explicit propositions.** Every label must correspond to a printed Lean obligation.
- **Three-valued search.** PROVED, REFUTED, and UNKNOWN are distinct; tactic failure is never a refutation.
- **Mode separation.** Logical subsumption, contextual adaptation, and domain-specific comparison are different relations.
- **Certificates over scores.** Heuristics may choose an alignment, but logical claims require closed proof objects.
- Background-relative results. The allowed proof environment is explicit and dependencies are recorded.

1. Formal Core: Component-Wise Subsumption

For the simplest aligned case, write the two theorems as:

```
T1 : forall x, A1 x -> C1 x
T2 : forall x, A2 x -> C2 x
```

`T1` has no stronger requirements than `T2` when anything satisfying `T2`'s requirements also satisfies `T1`'s:

$$\textit{forall x, A2(x) -> A1(x)}$$

`T1` has no weaker guarantee than `T2` when its conclusion entails the target conclusion in the explicitly shared ambient context:

$$\textit{forall x, C1(x) -> C2(x)}$$

Requirements are therefore contravariant; guarantees are covariant. For a fixed alignment and fixed comparison background, define `T1` to subsume `T2` when both of those directions are PROVED.

Forward status	Reverse status	Certified interpretation
PROVED	PROVED	Equivalent for this fixed relation/alignment.
PROVED	REFUTED	Strict in the forward direction for this fixed relation/alignment.
PROVED	UNKNOWN	Forward relation proved; strictness unsettled.
UNKNOWN	UNKNOWN	Unsettled.

Forward status	Reverse status	Certified interpretation
REFUTED	REFUTED	Incomparable for this fixed relation/alignment.

A global strictness claim is stronger than strictness under one heuristic alignment. If multiple admissible alignments or transports exist, strictness is global only when the alignment policy is unique/exhaustive or every admissible reverse route has been certified impossible.

Critical correction. “Not proved” is not “refuted.” A timeout, exhausted tactic portfolio, or ambiguous alignment must remain UNKNOWN.

2. Worked Example: Strict Arithmetic Subsumption

```
theorem T1 (x : Int) (h : x > 0) : x >= 1 := by
  omega

theorem T2 (x : Int) (h : x > 10) : x >= 0 := by
  omega
```

The forward component obligations are easy:

```
example (x : Int) : x > 10 -> x > 0 := by
  omega

example (x : Int) : x >= 1 -> x >= 0 := by
  omega
```

Strictness is not inferred merely because reverse proof searches fail. The comparator must also seek certified reverse refutations. Concrete witnesses suffice here: `x = 1` refutes `x > 0 -> x > 10`, and `x = 0` refutes `x >= 0 -> x >= 1`. A Lean implementation may encode these as proofs of the negated universal implications.

```
example : not (forall x : Int, x > 0 -> x > 10) := by
  intro h
  have hx := h 1 (by omega)
  omega

example : not (forall x : Int, x >= 0 -> x >= 1) := by
  intro h
  have hx := h 0 (by omega)
  omega
```

Under the fixed positional alignment `x <-> x`, `T1` therefore strictly subsumes `T2`: forward subsumption is proved and reverse subsumption is refuted.

3. Context-Free Guarantee Strength vs Contextual Adaptation

There are two related but semantically different notions that can be informally described as a “stronger conclusion.” They must be named and reported separately.

3.1 Context-free guarantee comparison

Given only the declared ambient/shared context `Delta`, compare the guarantees by:

Delta |- C1 -> C2

When this is well-typed and PROVED, the label “`C1` is at least as strong as `C2`” is justified relative to `Delta`. Target-specific assumptions are not available.

3.2 Contextual adapter obligation

After constructing a requirement transport `phi : Gamma2 => Gamma1`, one may instead ask:

Delta, Gamma2, h : C1[phi] |- C2

This is a legitimate and useful adapter judgement, but it is weaker than context-free guarantee comparison because the target requirements may help prove `C2`.

```
-- The premise P helps build the target conclusion.
example (P Q : Prop) (source : P -> Q) : P -> P /\ Q := by
  intro hP
  exact And.intro hP (source hP)
```

Here a source result `Q` adapts to `P /\ Q` in a context that already contains `P`. That does not prove the context-free implication `Q -> P /\ Q`. The report must therefore label this as contextual adaptation, not conclusion strengthening.

3.3 Target-theorem trivialization

If unrestricted proof search is allowed to use the already-established target theorem, a contextual goal can become vacuous:

```
example (A C1 C2 : Prop) (target : A -> C2) : A -> C1 -> C2 := by
  intro hA _
  exact target hA
```

The proof is correct but supplies no evidence that `C1` contributes to `C2`. The comparison background should therefore be explicit, the compared theorem declarations should be excluded from component proof search by default, and every dependency should be recorded. Excluding the target declaration does not make contextual comparison context-free; the two modes remain semantically distinct.

4. Theorem Telescopes, Alignment, and Requirement Transport

General Lean theorem types are dependent Pi telescopes, not merely a list of propositions followed by a conclusion:

```
T1 : forall {α : Type} [Preorder α]
       (f : α -> α) (hmono : Monotone f), C1 f

T2 : forall {α : Type} [LinearOrder α]
       (f : α -> α) (hstrict : StrictMono f), C2 f
```

The correct generalization of “weaker conditions” is therefore a dependent context transformation. After a shared input alignment `Delta` is selected, the comparator attempts to construct each remaining requirement of `T1` from the instantiated context of `T2`, in dependency order:

phi : Gamma2 => Gamma1

For example, a `LinearOrder alpha` instance can synthesize a parent `Preorder alpha`, and automation may derive `Monotone f` from `StrictMono f`. This is a constructive transport, not merely a conjunction implication.

4.1 Explicit binder roles

Binder syntax is not a complete semantic role system. A proposition-valued binder is often an assumption, but a non-propositional certificate object may also be a genuine applicability requirement. The comparator therefore exposes binder roles:

```
inductive BinderRole
  | sharedInput
  | requirement
```

```
  | witness
  | instanceRequirement
```

Automatic role inference is permitted as a convenience, but ambiguous classifications are printed and may be overridden. A material ambiguity should yield UNKNOWN rather than silently selecting the most favorable interpretation.

4.2 Alignment modes

Item	Meaning
Positional MVP	Align a common prefix of shared inputs by position, accepting types up to definitional equality.
Heuristic	Match by dependency shape, binder information, and definitional equality with bounded backtracking.
Explicit	User supplies the mapping when interfaces differ materially; this is the preferred escape hatch for difficult APIs.

Names are advisory only. Alpha-renaming must not affect the logical judgement.

5. Proof Background, Dependencies, and Derived Adapters

Every comparison is relative to an allowed background `B`: the declarations, instances, simplification lemmas, and domain procedures that proof search may use. A proof certificate validates exactly the proposition proved under that background; it does not validate a stronger informal story about why the proof succeeded.

```
structure ComparisonBackground where
  allowedDecls   : ...
  forbiddenDecls : ...
  options        : ...
```

By default, component comparison should operate on the elaborated types of `T1` and `T2` while forbidding direct use of those theorem declarations. This prevents the most obvious target-theorem shortcut. Equivalent library facts may still make an obligation easy, which is why dependency reporting remains essential.

5.1 Do not blindly search whole-theorem implication

A default search for `T1 -> T2` is not discriminating when `T2` is already a closed theorem in the environment:

```
example (P Q : Prop) (target : Q) : P -> Q := by
  intro _
  exact target
```

Blind whole-theorem implication is therefore excluded from the default subsumption classifier. When a theorem adapter is wanted, construct it compositionally from the certified requirement transport and the selected guarantee/adaptation proof. An optional adapter-discovery command may still exist, but it must use an explicit background and report dependencies.

5.2 Source use is a diagnostic, not the semantics

Whether a generated proof term syntactically depends on the source guarantee is useful diagnostic information. It is not itself a proof that the relationship is meaningful. The report therefore records `usesSourceResult`, `usesTargetDecl`, and the complete dependency set, but does not define logical strength in terms of syntactic source occurrence.

6. Three Comparison Modes

Mode	Core obligation	Interpretation
Logical subsumption	Gamma2 => Gamma1 and Delta |- C1 -> C2	T1 has no stronger requirements and no weaker guarantee.
Contextual adapter	phi : Gamma2 => Gamma1 and Delta,Gamma2,C1[phi] |- C2	A T1 result can be adapted into the T2 result under target requirements.
Domain comparator	Plugin-defined normalized semantic obligation	Compare the underlying quantity/contract rather than already-proved theorem propositions.

The user-facing command may show all applicable modes, but the report must never collapse them into one undifferentiated “stronger theorem” label.

7. Domain-Specific Semantic Comparison: Numerical Bounds

For upper-bound theorems, comparing two already-proved propositions about the actual quantity can be misleadingly easy. A more informative comparator abstracts the measured quantity and compares the bounds themselves.

$$(\forall h : \mathrm{Nat},\ h \le U1(f) \to h \le U2(f)) \iff U1(f) \le U2(f)$$

This obligation distinguishes the promised numerical bounds independently of the fact that either head-bound theorem already exists. It suggests a plugin interface for semantic comparators:

```
class SemanticComparator where
  recognizes : Expr -> MetaM Bool
  normalize  : Expr -> MetaM NormalizedContract
  compare    : NormalizedContract -> NormalizedContract -> MetaM ComparisonReport
```

Natural early domains include upper/lower bounds, norm bounds, cardinality bounds, error tolerances, and monotone cost measures. For theorem-library curation, these comparators may be more informative than raw proposition implication.

8. Certification Boundary

A tactic success flag or an in-memory `Expr` is not yet the final notion of certification. The implementation must close the proof object and cross Lean's normal checking boundary.

1. Proof search produces a candidate expression for the exact generated obligation.
2. Instantiate metavariables and reject any unsolved metavariables or synthetic placeholders.
3. Abstract the local context to obtain a closed theorem type and proof term.
4. Insert or otherwise check the declaration through Lean's normal declaration/kernel checking path.
5. Record declaration dependencies and axiom usage.
6. Only then mark the comparison edge CERTIFIED / PROVED.

Certification rule. “PROVED” in the public report should mean a closed proof artifact has been accepted by Lean, not merely that a tactic returned without error.

9. PoC Command and Report

```
#compare_theorems T1 T2
  (alignment := .positional)
  (mode := .all)
  (heartbeat := 200000)
```

Illustrative output:

```
ALIGNMENT
  shared: x : Int <-> x : Int
  roles : x=shared, h=requirement

REQUIREMENTS
  T2 => T1 : PROVED   [certificate req_2_to_1]
  T1 => T2 : REFUTED  [certificate not_req_1_to_2]

CONTEXT-FREE GUARANTEE
  C1 => C2 : PROVED   [certificate guar_1_to_2]
  C2 => C1 : REFUTED  [certificate not_guar_2_to_1]

COMPONENT-WISE RESULT
  T1 strictly subsumes T2 under this alignment : CERTIFIED

CONTEXTUAL ADAPTER
  T1 -> T2 under Gamma2 : PROVED
  uses source result    : yes
  dependencies          : [Int.order, ...]

CERTIFICATION
  closed / kernel checked : yes
  axioms                  : [...]
```

If a reverse direction is UNKNOWN, the output must say “subsumes; strictness unknown,” not “strictly subsumes.” If a context-free guarantee is ill-typed without additional abstraction, report NOT-WELL-FORMED and optionally offer the contextual adapter result instead.

10. Internal Data Model

```
inductive ProofStatus where
  | proved
  | refuted
  | unknown

structure BinderView where
  fvar       : Expr
  type       : Expr
  binderInfo : BinderInfo
  role       : BinderRole

structure Attempt where
  status       : ProofStatus
  certificate? : Option Name
  dependencies : Array Name
  axioms       : Array Name
  tacticTrace?  : Option String

structure ComparisonReport where
  alignment        : Alignment
  req_2_to_1       : Attempt
  req_1_to_2       : Attempt
  guarantee_1_to_2 : Attempt
  guarantee_2_to_1 : Attempt
  adapter_1_to_2?  : Option Attempt
  adapter_2_to_1?  : Option Attempt
```

The exact records may differ, but every proved/refuted edge should be tied to a closed certificate or reproducible generated declaration.

11. Core Algorithm

```
compare(T1, T2):
  ty1 <- typeOf(T1)
  ty2 <- typeOf(T2)

  tel1, C1 <- openTelescope(ty1)
  tel2, C2 <- openTelescope(ty2)

  roles1, roles2 <- classifyBinders(tel1, tel2)
  alignment <- alignSharedInputs(tel1, tel2, roles1, roles2)

  req_2_to_1 <- buildTransport(source=tel2, target=tel1, alignment)
  req_1_to_2 <- buildTransport(source=tel1, target=tel2, reverse(alignment))

  guar_1_to_2 <- proveContextFreeGuarantee(C1, C2, alignment, background)
  guar_2_to_1 <- proveContextFreeGuarantee(C2, C1, reverse(alignment), background)

  adapter_1_to_2 <- deriveAdapter(req_2_to_1, C1, C2, targetContext)
  adapter_2_to_1 <- deriveAdapter(req_1_to_2, C2, C1, sourceContext)

  certifyEverySuccessfulEdge()
  return classifyWithoutCollapsingUnknown()
```

11.1 Requirement transport

Process target requirements in dependency order. For each required binder:

1. Reuse an explicitly aligned shared input when available.
2. Search current locals for a term whose type is definitionally equal to the requirement.
3. For instance-implicit requirements, try typeclass synthesis.
4. For proposition-valued requirements, create a proof goal and run the bounded proof portfolio.
5. For non-propositional witnesses not classified as shared inputs, use configured constructors/search or return UNKNOWN.
6. Add the constructed term to the target instantiation before processing later dependent binders.

This preserves telescope dependencies and is more faithful than flattening arbitrary theorem contexts into a conjunction.

12. Proof-Search Portfolio

Search should be bounded, deterministic under fixed options, and ordered from cheap to expensive. A practical portfolio is:

1. Definitional equality and exact local assumptions.
2. Typeclass synthesis for instance requirements.
3. Simplification and deterministic local reasoning.
4. `grind` as a general-purpose engine where appropriate.
5. Theory-specific solvers such as arithmetic procedures.
6. Optional Mathlib or project tactics (`aesop`, `linarith`, domain-specific hooks).

Each attempt records the tactic path, resource budget, and dependencies. A proof-search failure remains UNKNOWN unless a separate sound procedure constructs a proof of the negated obligation.

13. MVP Scope

- **Alignment.** Common shared-input prefix, positional, types matched up to definitional equality.
- **Roles.** Instance-implicit and proposition-valued binders default to requirements; explicit overrides supported.
- **Relations.** Both requirement directions, both context-free guarantee directions, and derived contextual adapters.
- **Statuses.** PROVED / REFUTED / UNKNOWN, with strictness only from certified reverse refutation.
- **Background.** Compared theorem declarations forbidden by default in component search; dependencies always recorded.
- Certification. Successful edges materialized as closed, kernel-checked declarations with recorded axiom dependencies.
- **Verbose mode.** Print full contexts, input maps, generated obligations, and certificate names.

The MVP relies on standard Lean metaprogramming facilities: telescope opening, definitional equality, typeclass synthesis, tactic evaluation, and command-to-meta bridging. An implementation should pin and test against the exact Lean version used by its project rather than rely on moving documentation or unspecified API behavior.

14. Evaluation and Curation Evidence

Evaluate on curated theorem families with known relationships: arithmetic bounds, monotonicity lemmas, order/algebra hierarchy variants, stronger/weaker side conditions, deliberately incomparable pairs, and cases with multiple plausible alignments.

Item	Meaning
Soundness	Every positive/negative logical claim has a Lean certificate; no tactic failure is counted as refutation.
Coverage	Fraction of curated relations settled versus UNKNOWN.
Alignment quality	Number of heuristic alignments requiring correction or explicit override.
Cost	Runtime / heartbeats per attempted relation and per corpus cluster.
Diagnostics	Whether dependency reports reveal trivial or background-driven proofs.

If the comparator is used for theorem-library curation, emit a machine-readable selection manifest containing theorem pairs, alignment, roles, background policy, generated obligations, statuses, certificate names, dependencies, tool version, and search budget. The design document itself is not evidence that such a selection policy was executed.

15. Library-Scale Use

At corpus scale, comparisons form a directed graph. Certified component-wise edges can identify dominated lemmas; adapter edges can support theorem discovery; domain-comparator edges can identify genuinely tighter numerical or semantic guarantees.

To avoid quadratic search over an entire library, first cluster candidates by namespace, head symbols, normalized telescope fingerprints, result-type skeletons, and registered semantic domains. Run expensive semantic checks only inside plausible clusters.

Subsumption is not a total ranking of mathematical or research value. Incomparable statements may encode separations, limitations, different methods, or useful specialized interfaces. The comparator should support curation, not replace scientific judgement.

16. Roadmap

Item	Meaning
Phase 1	Positional alignment; explicit role overrides; tri-state/refutation semantics; context-free guarantee mode; derived adapters; closed certificates; dependency report.
Phase 2	Bounded structural matching for reordered inputs; richer dependent transports; better background policies; proof caching.
Phase 3	Domain comparator plugins; certified counterexample/refutation backends where available; corpus indexing and selection manifests.
Phase 4	Library tooling: find subsumed lemmas, find stronger guarantees, detect accidental duplicate APIs, and CI linting.

17. Conclusion

Certified theorem subsumption should not be framed as asking automation which theorem is “better” or “stronger” in the abstract. The robust design is to select a precise relation, print the exact obligation, search within an explicit background, close and kernel-check the resulting proof, preserve UNKNOWN, and report only the label warranted by that proposition.

With those constraints, Lean metaprogramming is a strong fit: metaprogramming supplies the telescope inspection, alignment, goal construction, tactic orchestration, and certificate plumbing; tactics supply incomplete but useful search; domain plugins supply semantically appropriate normalization; and the kernel supplies trust.

> Design principle. Automate discovery. Certify exact propositions. Separate comparison modes. Preserve uncertainty. Record dependencies.

References and Implementation Notes

Lean Language Reference: Tactic reference and core language documentation. https://lean-lang.org/doc/reference/latest/

Metaprogramming in Lean 4: MetaM, telescope opening, and definitional equality. https://leanprover-community.github.io/lean4-metaprogramming-book/main/04_metam.html

Lean.Meta.SynthInstance: Typeclass synthesis APIs. https://lean-lang.org/doc/api/Lean/Meta/SynthInstance.html

Lean.Elab.Tactic.Basic: Programmatic tactic evaluation. https://lean-lang.org/doc/api/Lean/Elab/Tactic/Basic.html

Lean.Elab.Command: Command elaboration and lifting into term/meta elaboration. https://lean-lang.org/doc/api/Lean/Elab/Command.html

Lean AddDecl source: Relevant declaration insertion / checking boundary for a pinned implementation. https://github.com/leanprover/lean4/blob/v4.31.0/src/Lean/AddDecl.lean